

Pure vs Impure Functions

Pure function: Always predictable, like a calculator. Input → Output, nothing else changes.

Impure function: Has side effects, like updating a global spreadsheet while calculating something.

// Pure

```
const add = (a,b) => a+b;
```

// Impure

```
let counter = 0;
```

```
const increment = () => counter++;
```

// each time new output

Q. Why prefer pure functions in React/Redux.

Because pure functions don't mutate state, React can reliably detect updates and Redux can track state history, making the app predictable and easier to debug

Q. What Are **Side Effects** in JavaScript?

A **side effect** happens when a function **does something other than returning a value** —

it **changes something outside itself** or **interacts with the outside world**.

1 Modifying a Global Variable

```
let count = 0;
```

```
function increment() {  
  count++; // ✗ Side effect (mutates external state)  
}
```

Why it's bad:

- Anyone calling increment() secretly changes global state.
- Hard to track bugs.

✓ Pure version:

```
function increment(count) {  
  return count + 1;  
}
```

2 Changing an Object / Array (Mutation)

```
function addItem(arr, item) {  
  arr.push(item); // ✗ Mutates original array  
}
```

This changes the original reference → breaks React state tracking.

✓ Pure version:

```
function addItem(arr, item) {  
  return [...arr, item];  
}
```

3 DOM Manipulation

```
function updateTitle() {  
  document.title = "Hello"; // ✗ Directly touches browser DOM  
}
```

This interacts with the outside world → side effect.

✓ In React:

This belongs in useEffect, not inside render logic.

4 API Calls / Network Requests

```
function fetchUsers() {  
  return fetch("/api/users"); // ✗ External IO  
}
```

Fetching data changes nothing locally but **depends on outside systems** → side effect.

That's why Redux uses **middleware** (Thunk/Saga) to isolate this.

5 Logging to Console

```
function calculate(a, b) {  
  console.log("Calculating..."); // ✗ Side effect  
  return a + b;  
}
```

Even logging counts — it affects the outside environment.

Pure version:

```
function calculate(a, b) {  
  return a + b;  
}
```

(Log outside if needed.)

6 Reading Current Time / Randomness

```
function getDiscount() {
```

```
return Math.random() * 10; // ✗ Not predictable
}
```

OR

```
function now() {
  return new Date(); // ✗ Depends on system clock
}
```

These make the function **non-deterministic**.

✓ Pure approach: pass values as arguments.

```
function getDiscount(randomValue) {
  return randomValue * 10;
}
```

7 Writing to LocalStorage / Files

```
function saveUser(user) {
  localStorage.setItem("user", JSON.stringify(user)); // ✗ Side effect
}
```

This changes persistent state outside JS memory.

A side effect is any operation where a function modifies external state, performs IO, mutates data, or depends on something outside its inputs, making the function impure and less predictable.

✓ Relation Between Hooks and Side Effects (Short Answer)

React Hooks are used to manage side effects in a controlled way.

Specifically, `useEffect` lets us run side-effect code (like API calls, DOM updates, timers, subscriptions) **outside the rendering logic**, keeping components predictable and pure.

